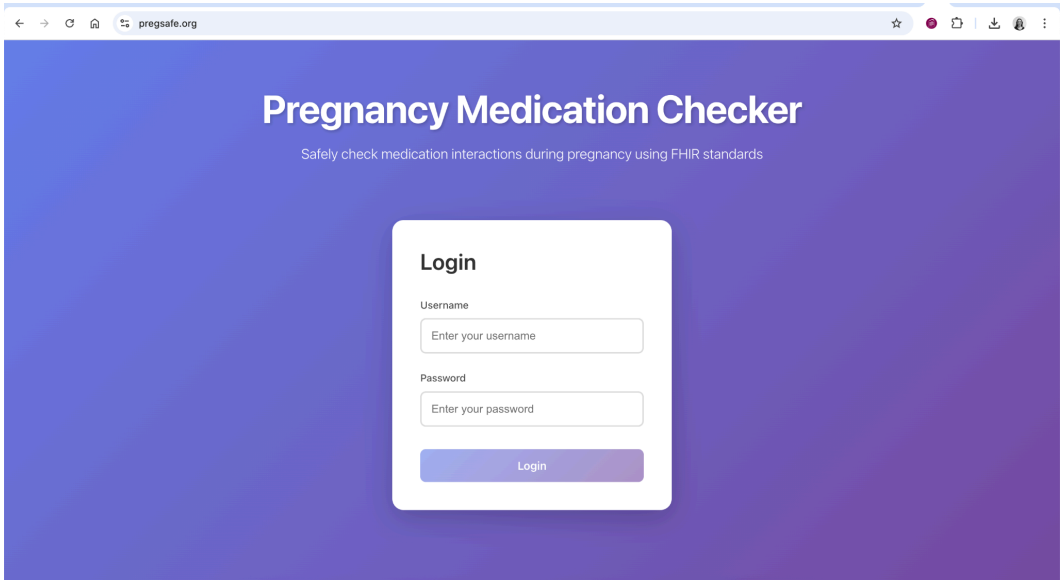
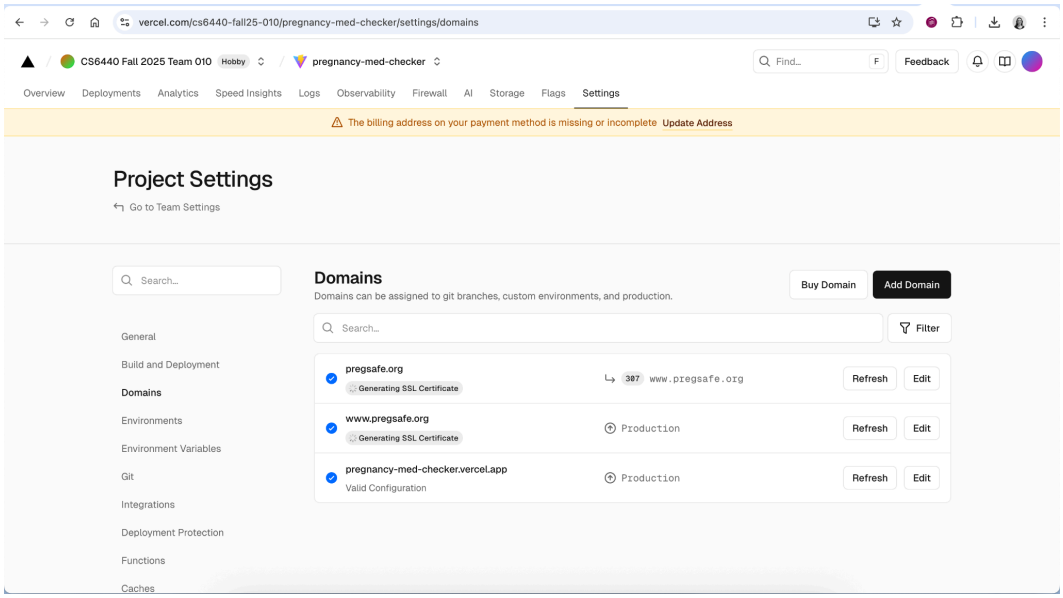
Final Project Sprint 4:
CS 6440

Valentina Cano Arcay, Lana P kareem, Faezeh Goli
varcay3@gatech.edu, lkareem6@gatech.edu, faezeh@gatech.edu

1 DEPLOYMENT

Deployed frontend to pregsafe.org, using Vercel guidelines: https://github.gatech.edu/cs6440-students/vercel-deployment-guide





2 DEMO

Here is the link to the demo video located on Google Drive:

[Sprint 4 Demo Video]

3 ACCOMPLISHMENTS/ TASKS COMPLETED

3.1 Faezeh

Week 1 (8.5 hours):

Set up Docker to build and start all containers and conducted local testing for both the frontend and backend. After encountering some issues with Docker Compose, I proceeded to run the backend locally using the most recent updated repository, which executed successfully. I then established a connection to RxNorm and tested it locally with a sample medication name, the API returned the expected JSON response containing the drug name and its corresponding RxCUI.

Worked on adding our planned features to the RxNorm integration, including autocomplete, drug name to RxCUI mapping, and retrieval of drug properties. During the implementation of the autocomplete feature, I explored several approaches to improve result accuracy. Initial methods, such as Approximate

Term and Spelling Suggestions, did not provide the desired output, so I shifted to using the Get Drug endpoint to perform a broader search. This method returned more complete results, though it occasionally included duplicates and longer lists. To address this, I implemented logic to detect and remove duplicates and to shorten the returned list to a manageable length.

In addition to the technical work, I also contributed to our project sprint documentation by creating the initial draft and recording my progress updates.

Week 2 (14 hours):

In the second week leading up to Sprint 4, I focused on the OpenFDA integration. My primary tasks involved establishing a connection to the OpenFDA API and integrating the OpenFDA Drug Label API into the backend. In addition to implementing a utility that identifies and extracts relevant text sections from FDA label data, I also integrated asynchronous API calls, response caching, and CORS-enabled FastAPI configurations. These enhancements were aimed at improving performance through short term caching and ensuring secure communication between the backend and frontend.

- Backend Setup & Environment Configuration

  Created and configured .env and requirements.txt files for the OpenFDA connector module. Verified environment loading and app initialization.

- FastAPI App Configuration (main.py)

  Developed the main entry point for the backend, initializing FastAPI app, configuring CORS middleware for frontend communication, and mounting API routes for OpenFDA integration.

- Schema Definition (schemas.py)

  Defined structured response models using Pydantic, ensuring API responses (from OpenFDA) are validated and follow a consistent JSON schema. Added LabelResponse and EventResponse models.

- Data Extraction Utility (utils.py)

Implemented a utility function to parse and extract relevant pregnancy and lactation information from OpenFDA's complex JSON responses. Added logic to handle varying key names and fallback searches in text sections.

- OpenFDA Client Logic (openfda_client.py)

  Built an asynchronous HTTP client using httpx to communicate with OpenFDA endpoints (/drug/label.json, /drug/event.json). Added caching logic (TTL-based dictionary) to improve performance and reduce API call frequency.

To integrate the files mentioned above, tools and libraries including FastAPI, HTTPX, Uvicorn, Pydantic, Python-dotenv, and OpenFDA API were used.

At this stage, this integration empowers our backend to fetch, process, and deliver FDA verified safety data for medications. This integration lays the foundation for displaying pregnancy and lactation warning on the frontend, potentially expanding into adverse event visualization, and increasing search functionality if combined with RxNorm.

3.2 Lana

My goal for this sprint was to finalize the connection of the backend and frontend from the last sprint, design the UI for the frontend and create the proper functionality needed for the main dashboard of the frontend, setup the PubMed API connection, and create the demo video to display our current progress. Each week, I split up my time across 10 hours for a little over 20 hours this sprint, with the following breakdown of tasks:

- Backend & Frontend connection investigation (1.5 hours)
  - Researched the failure for the backend and frontend to connect from being unable to successfully setup the connection during last sprint, and ended up finding that having a VPN turned on while fetching the installation for Vite does not work. So, I turned off the

VPN and reran the installation, and thus had to reconfigure the temporary frontend setup that I created in the last sprint.

- Refactor Frontend code setup after determining backend and frontend connection issue (2 hours)
    - Last sprint's frontend code was just a temporary layout until I resolved the issue with the connection, so once I solved that, I purged all the frontend setup and created all new code for the following files:
        - Package.json: updated dependencies
        - vite.config.ts: removed API proxy and simplified to basic Vite configuration
        - App.tsx: changed from API health check to default Vite template
        - TypeScript config files: tsconfig.app.json, tsconfig.node.json
        - CSS files: App.css, index.css
        - Dockerfile.backend: updated dockerfile to route and deploy properly for the backend, also added make command support for building the application
- Frontend/Backend Authentication System, Login UI, Provider/Patient Dashboard (13 hours)
    - Authentication system:
        - backend/app/auth.py: added JWT authentication for user authentication functions, token generation and validation, and created two accounts for different roles (provider/patient)
            - Provider account information:
                - Username: provider
                - Password: provider123
                - Role: Provider (Dr. Sarah Johnson)
            - Patient account information:
                - Username: patient

        - Password: patient123
        - Role: Patient (Jane Doe)
    - main.py: added authentication endpoints
        - POST /api/auth/login
        - GET /api/auth/me
        - GET /api/auth/test-accounts
    - backend/pyproject.toml: added dependencies
    - backend/uv.lock: updated dependency lock file
- Frontend Design:
    - frontend/src/App.tsx: Updated to use HomePage component with login flow
    - frontend/src/App.css: Added styling for login page, dashboards, and components
    - frontend/src/components/ErrorBoundary.tsx: React error boundary for error handling
    - frontend/src/components/FHIRStatus.tsx: Component to display FHIR connection status with auto-refresh
    - frontend/src/components/Header.tsx: Header with user info, FHIR status, and logout button
    - frontend/src/components/HomePage.tsx: Main landing page with login form and role-based routing
    - frontend/src/components/LoginForm.tsx: Login form component with validation
    - frontend/src/components/PatientApp.tsx: Patient dashboard component
    - frontend/src/components/ProviderApp.tsx: Provider dashboard component
    - frontend/src/index.css: Updated global styles
    - frontend/src/types/auth.ts: TypeScript types for User, UserRole, LoginResponse
    - frontend/src/utils/auth.ts: Authentication utilities:
        - Login/logout functions

            - Token management (localStorage)
            - Authenticated API call helpers
        - frontend/vite.config.ts: Added API proxy configuration for development
- PubMed API setup (2.5 hours)
    - backend/app/main.py: Added PubMed router import and registration
    - backend/app/pubmed.py : FastAPI router with endpoints:
        - POST /api/publications: Search PubMed for publication IDs
        - GET /api/publications/details: Get detailed publication information
    - backend/services/pubmed_integration/__init__.py: Package initialization
    - backend/services/pubmed_integration/pubmed_service.py: Service functions:
        - search_publications(): Search PubMed API
        - get_publication_details(): Parse XML and extract publication data
- Code Reviews / Refactoring some team's code (2 hours)
    - Updated CORS to allow https://www.pregsafe.org and https://pregsafe.org in addition to localhost connection
    - Vercel deployment:
        - deploy.sh: made VERCEL_TOKEN optional (works with or without token)
        - frontend/vercel.json: Vercel configuration for frontend build
        - Vercel.json: root-level Vercel configuration pointing to frontend directory
    - RxNorm Refactoring
        - backend/app/main.py: updated to import RxNorm router from new location

        - backend/app/rxnorm.py: FastAPI router moved from service layer
        - backend/services/rxnorm_integration/__init__.py: Package initialization
        - backend/services/rxnorm_integration/rxnorm_service.py: moved/refactored from rxnorm_integration/rxnorm.py and separated service functions from FastAPI code
- Demo Video and Sprint 4 document (3 hours)
    - Documented task breakdown
    - Created the demo script and demo video to explain progress made on the project thus far

3.3 Valentina

My goal for this sprint was to refactor the FHIR integration service, investigate Synthea data generation for realistic pregnancy-focused patient data, and set up a local FHIR server for development and testing. Over approximately 20 hours, I completed the following work:

- Synthea Data Investigation (2 hours):
    - Researched Synthea synthetic data generation tool and its module system
    - Investigated existing pregnancy-related modules and identified gaps for our use case
    - Explored Synthea's FHIR resource generation capabilities and output formats
    - The documentation I viewed was the following: https://github.com/syntheticheath/synthea/wiki/Basic-Setup-and-Running
- FHIR Client Library Migration Research (2 hours):
    - Investigated issues with the SMART on FHIR client library (`fhirclient`) that was previously implemented
    - Researched alternative FHIR client libraries and evaluated `fhir-py` as a simpler, more reliable solution

- FHIR Local Server Setup (3 hours):
    - Encountered persistent data quality issues with public FHIR servers
    - Researched local FHIR server options and decided to set up a local HAPI FHIR server for more reliable testing
    - Added the `hapiproject/hapi:latest` Docker image to `docker-compose.yml` for local development
    - Configured environment variables for FHIR server URL and settings
    - Tested server connection and health checks and added to Makefile for easier configuration
    - Trouble shooted unexpected issues with Docker on my computer
- Code Architecture Refactoring (2.5 hours):
    - Due to some disorganization in the code, planned and implemented domain-driven design (DDD) architecture for better code organization and refactored `backend/services/fhir_integration/`
    - For more details, see PR description: https://github.gatech.edu/lkareem6/CS6440-Final-Project/pull/5
- FHIR Client Migration to fhir-py (2.5 hours):
    - Implemented new FHIRClientService using `fhir-py` for simple HTTP-based FHIR operations
    - Updated all CRUD and search operations to use the new client
    - Tested connection and basic operations with the local HAPI FHIR server
- Synthea Custom Module Development (2.5 hours):
    - Created custom Synthea modules for pregnancy-focused data:
        - `pregnancy_ga.json`: Gestational age tracking throughout pregnancy
        - `pregnancy_exposures.json`: Medication exposures during pregnancy with timing information

        - `lactation.json`: Lactation-related data and medication safety during breastfeeding
    - Configured Synthea generation script (`run_synthea.sh`) with custom modules
    - Generated 30,000 female patients aged 18-40 for testing (data excluded from git via `.gitignore`)
- Synthea Data Formatting Investigation (1 hour):
    - Investigated Synthea-generated FHIR resource formats
    - Identified challenges with mapping Synthea output to our ingestion requirements
    - Began working on parsing logic for pregnancy and medication-focused FHIR resources
    - Note: Data ingestion is still in progress due to formatting complexities
- Vercel Project Creation and Application Deployment (2 hours):
    - Researched deployment methods and platforms suitable for our application stack; decided to follow the process in https://github.gatech.edu/cs6440-students/vercel-deployment-guide
    - Created a Vercel account and set up a project and token
    - Selected a custom domain: pregsafe.org
    - Initialized Vercel project in the `pregnancy-med-checker` directory; ran `vercel` command for initial setup and linking
    - Used automated deployment script .deploy.sh and added to Makefile
- Code Review (1.5 hours):
    - Reviewed teammates' code contributions and pull requests
    - Ensured code quality and consistency across the project
- Sprint Document (1 hour):
    - Documented completed tasks and time breakdown
    - Wrote challenges section and sprint plans for next period

4 CHALLENGES

4.1 Faezeh

During the RxNorm integration, I explored several approaches to enhance the accuracy and consistency of the autocomplete feature. I experimented with both the spelling method and the approximateTerm function. After refining the logic, I combined these methods to support fuzzy matching, covering typos and partial inputs, and increased the result cap to 10. Additionally, I implemented duplicate removal to generate a cleaner and more relevant list of suggestions.

During the OpenFDA integration, I faced "Import Path Error ", which happened due to missing package initialization. By adding __init__.py files to my app/ directory, I marked the directories as a package and the issue was resolved.
I also got "OpenFDA 404 Errors" when requesting "https://api.fda.gov/drug/label.json?" for the sample term Tylenol. This happened because the API had no label data or the query data was invalid. I tried to debug the query string encoding, by verifying the endpoint syntax from OpenFDA documentation, and added a graceful error handling for scenarios with no results.
Another challenge working with OpenFDA was handling inconsistent field names. To help manage this characteristic of the API, I created a flexible parser utils.py, that searches multiple possible keys and fallback text search within the label field.
One concern working with OpenFDA PI was performance for repeated queries. Each query makes a live call which may slow down the app for repeated terms. A TTL based local cache (in memory dictionary) was implemented in the openfda_client.py file, to reuse results for a short duration.

4.2 Lana

In regards to the challenges faced during this sprint, one of the challenges I faced was determining an effective design for the frontend UI display. I researched many different platforms that are similar in functionality where patients can

interact with their provider, such as MyChart, Athena, and other systems that use FHIR, and I found that having the FHIR data connection displayed somewhere on the screen is helpful for users to know where the system is retrieving clinical data from. I also had to ensure that the dashboard and login pages were visually pleasing and easily navigable to users, so this was a challenge figuring out the best format, which colors to use, and what information is important for users to view on each page. On top of that, I also am not very familiar with the Vite framework, so I had to do further research for each step in the code development process and work through bugs along the way.

For the PubMed API, it was difficult to understand the NCBI E-utilities API structure as well as handling XML parsing vs. JSON responses. I also worked through difficulties with managing different endpoints, such as esearch vs. efetch. I also had to make sure all the code changes I made aligned with our current code architecture, such as our implementations for FHIR and RxNorm in order to make sure that the code-base maintains consistency.

A learning curve that I faced was understanding FastAPI patterns, such as router patterns, dependency injection, and Pydantic models for validation. I also had to research Pydantic models themselves and what purpose they serve in the FastAPI world.

4.3 Valentina

- FHIR library: The previously implemented SMART on FHIR client library (`fhirclient`) was consistently producing bugs and always falling back to direct HTTP requests. After investigation, I decided to migrate to the `fhir-py` library, which provides simpler HTTP-based FHIR operations without the complexity of SMART authentication
- Docker Proxy Configuration Issues: While setting up the local HAPI FHIR server using Docker, I encountered unexpected issues related to proxy configuration on my computer. The proxy setup was interfering with Docker's network connectivity, preventing the container from properly connecting to external resources and causing connection failures. This required troubleshooting Docker's proxy settings, network configuration,

and environment variables to resolve the connectivity issues and deploy the local FHIR server.

- Synthea Data Formatting and Ingestion: While I successfully generated 30,000 synthetic patients using custom Synthea modules, I am still struggling with the Synthea data formatting and haven't been able to complete the ingestion process. The FHIR resources generated by Synthea need careful parsing and mapping to extract the relevant fields needed for pregnancy condition and medication checks. This work is ongoing and I plan to complete it this weekend or early next week.
- Synthea Module Learning Curve: Understanding how Synthea modules work required significant investigation time.
- Backend Deployment: during front end deployment to Vercel, we hit an architectural mismatch, because Vercel does not support deploying Docker images directly while our backend is packaged with Docker (Dockerfile.backend). We're evaluating two paths. First, adapt the backend to Vercel's serverless functions; this comes with trade-offs like slower first requests, disabled startup/shutdown hooks, and execution time limits. Second, host the backend on a Docker-friendly service and connect it to the Vercel frontend via API. This keeps our current Docker setup but means operating two platforms and configuring resource sharing and secrets across services. I also found that Vercel was native deployment for the FastAPI framework, so I will look into this further: https://vercel.com/docs/frameworks/backend

5 SPRINT PLANS

The group's plan for the next two weeks, leading up to the next project check-in on November 24, is to implement context-aware risk checking, which includes adding user context options such as pregnancy and lactation. Adjustments in the backend logic will be made to filter risks based on these contextual factors. The team also plans to introduce severity levels (low, medium, high) to categorize the identified risks. In addition, the clinician versus patient user workflows are scheduled for implementation.

Another planned task focuses on user interface (UI) and visualization enhancements. Drug interactions will be displayed with color-coded severity levels, and research excerpts or reference links will be added to each flagged interaction. The user interface will be further refined, and the patient selection interface for clinicians will be developed.

We also plan on investigating and integrating DrugBank as a data source for comprehensive drug interaction datasets and plan the integration approach for drug-drug, drug-food, and drug-supplement interactions.

Valentina will focus on completing the FHIR data ingestion pipeline by resolving Synthea data formatting challenges and implementing parsing logic to extract all relevant fields needed for pregnancy condition and medication checks. She will load the generated Synthea data onto the HAPI FHIR server and verify data accessibility. To improve performance, she will research and potentially implement an asynchronous version of the FHIR client service. Finally, she will work with the frontend to integrate FHIR patient data retrieval into the UI, implementing API endpoints for patient search and medication/condition retrieval to ensure FHIR data properly populates the medication interaction checker interface.

Faezeh will implement and test the openfda_router.py file to expose backend endpoints for frontend access. She will connect the OpenFDA module to the main application and verify that it responds correctly to user drug name searches during runtime. Additional steps may include displaying the retrieved data on the website, enhancing performance using Redis for faster data storage and retrieval, and implementing logic to handle missing or incorrect data by providing appropriate error messages.

Lana will continue designing and developing the UI for the frontend and incorporating more of the backend features to be displayed in the frontend, such as expanding on the provider/patient dashboards for their respective functionalities. In terms of provider-specific features, Lana will add patient search and medication checking, and in terms of patient-specific features, she will add functions to view their own medications and any observations the

provider or patient has throughout the pregnancy. For the frontend display of the PubMed API, Lana will add the frontend components to display PubMed research summaries, integrate PubMed results into medication safety checks, add caching for frequently searched queries, create UI for displaying research evidence alongside medication information, add filtering and sorting options for search results, and implement relevance scoring for search results. Additionally, I will begin implementing the visualization portion for displaying drug interaction risks.